

Interview Preparation Distributed Multi-Agent AI Knowledge Assistant (Agentic RAG)

Prepared as if by a senior interviewer from Oracle/Amazon/Google with 10+ years of hiring experience. Read this top to bottom. Every answer is written in **your voice**.

SECTION 1 PROJECT INTRODUCTION

Q1. Explain the Agentic RAG project end-to-end.

Your Answer: "This project is a production-grade AI assistant that answers questions from a knowledge base think of it like a smart document Q&A system. The key word is 'Agentic' unlike a simple RAG pipeline where you retrieve documents and generate an answer once, this system uses multiple specialized agents that coordinate in a stateful pipeline and can self-correct.

The four agents are: 1. **Planner:** Understands the user query and decides retrieval strategy 2. **Retriever:** Fetches relevant documents using hybrid search 3. **Generator:** Uses an LLM (Llama 3.1 via Groq) to generate the answer 4. **Grader:** Evaluates answer quality on faithfulness and relevance

If the Grader scores the answer below a threshold, it triggers re-retrieval a Reflexion-style self-correction loop. The system scored 0.97+ on faithfulness and relevance metrics. The backend is FastAPI with SSE-based streaming, and everything is containerized with Docker Compose and served via Nginx."

SECTION 2 RAG FUNDAMENTALS

Q2. What is RAG? Why is it better than fine-tuning?

Your Answer: "RAG stands for Retrieval-Augmented Generation. Instead of training an LLM on domain-specific data (which is expensive and the knowledge gets stale), RAG keeps documents in a separate knowledge base and retrieves relevant ones at query time, then feeds them to the LLM as context.

Advantages over fine-tuning: - **Knowledge is updatable:** Add new documents to the DB without retraining - **Cheaper:** No GPU training required - **Transparent:** You can see exactly which documents were retrieved - **Reduces hallucination:** The LLM is grounded in retrieved facts, not just its training weights - **Source attribution:** You can cite which document the answer came from

The limitation of RAG is retrieval quality if you don't retrieve the right documents, the LLM generates a bad answer regardless of how good it is. That's why my hybrid search + reranking pipeline was critical."

Q3. Explain the RAG pipeline step by step.

Your Answer: "The pipeline: 1. **Ingestion** (offline): Documents are chunked into segments, each chunk is embedded into a vector using SentenceTransformers, stored in ChromaDB. A BM25 index is also built over the same chunks. 2. **Query time:** - User sends a question - Planner agent analyzes query intent - Retriever agent runs hybrid search: ChromaDB vector search + BM25 keyword search, results are fused with weighted scoring - Cross-Encoder reranks top results - Generator agent sends retrieved context + query to Llama 3.1 via Groq API - LLM generates an answer with streaming tokens - Grader agent evaluates the answer - If score < threshold, loop back to retrieval with a refined query - If score passes, stream final answer to client via SSE"

SECTION 3 LANGGRAPH & AGENTS

Q4. What is LangGraph? Why not use LangChain directly?

Your Answer: "LangGraph is a framework for building stateful, multi-agent workflows as graphs. Each agent is a node, and edges define the flow between them, including conditional edges branches based on runtime decisions like 'did the Grader pass or fail?'"

LangChain is a chain linear sequence of steps. For simple Q&A it works, but it doesn't support cycles (loops) natively. My Reflexion loop requires cycling back from Grader Retriever when confidence is low. LangGraph supports this because it builds a directed graph, not a chain. The state is also explicitly managed as a typed schema that flows through all nodes each agent can read and update the shared state."

Q5. Explain each of your four agents.

Your Answer: "**Planner Agent:** Receives the raw user query. Its job is to analyze intent is this a factual question, a comparison, a definition? It may rewrite the query for better retrieval for example, expanding abbreviations or breaking a compound question into sub-queries. It sets the retrieval strategy in the shared state.

Retriever Agent: Executes the retrieval. It runs the hybrid search pipeline ChromaDB for semantic similarity, BM25 for keyword matching fuses scores, then applies Cross-Encoder reranking. Returns top-k most relevant chunks with metadata.

Generator Agent: Takes the retrieved context and original query, constructs a prompt, and calls the LLM API (Groq with Llama 3.1). Uses the provider-agnostic LLM factory can switch to Gemini without changing the agent code. Streams the response token by token.

Grader Agent: Evaluates the generated answer on two dimensions: faithfulness (is the answer supported by the retrieved documents?) and relevance (does it actually answer the question?). If either score drops below threshold, it sets a flag in state to trigger re-retrieval with a modified query."

Q6. What is a stateful pipeline in LangGraph?

Your Answer: "In LangGraph, the entire graph shares a typed state object a Python TypedDict or Pydantic model. Every node (agent) receives this state, reads what it needs, makes modifications, and returns the updated state. The state persists across all nodes in a single run. For example, my state includes:

```
class AgentState(TypedDict):
    query: str
    rewritten_query: str
    retrieved_docs: List[Document]
    generated_answer: str
    faithfulness_score: float
    relevance_score: float
    retry_count: int
    final_answer: str
```

The Planner writes `rewritten_query`, the Retriever writes `retrieved_docs`, the Generator writes `generated_answer`, the Grader writes the scores and sets a flag. The conditional edge after Grader reads the scores to decide: go to END or loop back to Retriever."

Q7. What is the Reflexion pattern? How did you implement it?

Your Answer: "Reflexion is a self-improvement pattern for LLM agents instead of accepting the first generated answer, the system reflects on whether it's good enough and retries if it's not. It's like a quality control loop.

In my implementation: 1. Generator produces an answer 2. Grader scores it on faithfulness and relevance (0 to 1 scale) 3. If score < 0.7 (threshold), Grader adds critique to state: 'Answer not supported by retrieved docs retrieved context may be insufficient' 4. The graph's conditional edge routes back to Planner/Retriever 5. Planner uses the critique to generate a better query 6. Retriever fetches new or different documents 7. Generator retries

I limit retries to 3 to prevent infinite loops. This is why I achieved 0.97+ scores the system self-corrects instead of accepting low-quality answers."

SECTION 4 HYBRID SEARCH & RERANKING

Q8. Explain vector search. What is an embedding?

Your Answer: "An embedding is a numerical representation of text in a high-dimensional vector space for example, a 384-dimensional vector for SentenceTransformers. The key property is: texts that are semantically similar will have vectors that are geometrically close (high cosine similarity).

Vector search works by: 1. At index time: each document chunk is embedded vector stored in ChromaDB 2. At query time: the query is embedded using the same model query vector 3. ChromaDB computes cosine similarity between query vector and all stored vectors 4. Returns top-k most similar chunks

This captures semantic meaning 'automobile' and 'car' are different strings but will have similar vectors, so vector search handles synonyms and paraphrase naturally."

Q9. What is BM25? How does it differ from vector search?

Your Answer: "BM25 (Best Match 25) is a classic IR algorithm based on TF-IDF with improvements for document length normalization and term saturation. It scores documents based on exact keyword matching how often query terms appear in the document, weighted by how rare those terms are across the corpus (IDF).

Key difference from vector search: - **BM25**: Exact term matching. Great for precise keywords, named entities, product codes, rare technical terms. - **Vector search**: Semantic matching. Great for paraphrase, synonyms, intent matching.

They're complementary. A query like 'What is LLM?' vector search finds semantically relevant docs. A query like 'error code ORA-01722' BM25 finds the exact error code better. That's why hybrid search outperforms either alone."

Q10. How does Hybrid Search work? Explain score fusion.

Your Answer: "I run both searches in parallel: - ChromaDB returns top-k docs with cosine similarity scores (0 to 1) - BM25 returns top-k docs with BM25 relevance scores (can be > 1, varies by corpus)

Then I apply weighted score fusion a technique called Reciprocal Rank Fusion or simple weighted linear combination:

```
# Normalize scores to [0,1]
vector_score_normalized = (score - min) / (max - min)
bm25_score_normalized = (score - min) / (max - min)

# Weighted combination
final_score = alpha * vector_score + (1 - alpha) * bm25_score
# alpha = 0.6 in my case (favor semantic)
```

Documents are re-ranked by final_score. This gives documents that score well on BOTH methods a higher rank they're likely both semantically relevant and contain the right keywords."

Q11. What is Cross-Encoder reranking? Why do you need it if hybrid search is already good?

Your Answer: "Bi-encoders (used in vector search) encode the query and document independently into vectors, then compare. This is fast but less accurate the model doesn't see the query-document interaction.

A Cross-Encoder takes the query AND document together as input and produces a single relevance score. It can see the interaction between query terms and document content, making it much more accurate. The model I used is ms-marco-MiniLM-L-6-v2 specifically trained for relevance scoring.

But Cross-Encoders are slow you can't run them on all documents in the corpus. So the pipeline is: 1. Retrieve top-50 candidates with fast hybrid search 2. Rerank top-50 using Cross-Encoder much more accurate ranking 3. Take top-5 for LLM context

This two-stage approach gives both speed and accuracy."

SECTION 5 FASTAPI & SSE STREAMING

Q12. Why FastAPI over Flask for this project?

Your Answer: "FastAPI is asynchronous by design built on Python's async/await and ASGI. This matters for two reasons in my project: 1. **LLM API calls are I/O bound:** While waiting for Groq to stream tokens, the server can handle other requests. With Flask's synchronous WSGI, one slow LLM request blocks the thread. 2. **SSE streaming:** FastAPI's StreamingResponse makes SSE trivial to implement. Flask needs workarounds for streaming. 3. **Automatic validation:** Pydantic v2 models define the API schema FastAPI auto-validates requests and generates OpenAPI docs. 4. **Performance:** FastAPI benchmarks show 3-5x throughput vs Flask for I/O-bound workloads."

Q13. What is SSE? How did you implement it?

Your Answer: "SSE (Server-Sent Events) is a protocol where the server keeps an HTTP connection open and pushes events to the client one by one. It's unidirectional (server client), unlike WebSockets which are bidirectional.

For LLM streaming, each token the LLM generates is sent as an SSE event the user sees the answer being written word by word, which feels much more responsive than waiting 5 seconds for the full answer.

Implementation in FastAPI:

```
from fastapi.responses import StreamingResponse

async def token_generator(query: str):
    async for token in llm.astream(query):
        yield f'data: {token}\n\n' # SSE format

@app.post('/chat')
async def chat(request: ChatRequest):
    return StreamingResponse(
        token_generator(request.query),
        media_type='text/event-stream'
    )
```

On the frontend, I use EventSource API in JavaScript to receive events."

Q14. What is Pydantic v2? How did you use it?

Your Answer: "Pydantic is a data validation library using Python type hints. Pydantic v2 is a complete rewrite in Rust it's 5-50x faster than v1."

I used it for: 1. **API request/response models:** Define the shape of incoming requests and FastAPI validates them automatically 2. **Agent state schema:** The LangGraph state is a Pydantic model with field validators 3. **Configuration:** Settings loaded from environment variables with type coercion

Example:

```
from pydantic import BaseModel, Field

class ChatRequest(BaseModel):
    query: str = Field(..., min_length=1, max_length=1000)
    max_retries: int = Field(default=3, ge=1, le=5)

class AgentState(BaseModel):
    query: str
    retrieved_docs: list[Document] = []
    faithfulness_score: float = 0.0
```

FastAPI uses these models to auto-generate OpenAPI/Swagger docs and validate all incoming data."

SECTION 6 DOCKER & NGINX

Q15. Explain your Docker Compose setup for this project.

Your Answer: "I use a multi-service Docker Compose setup: - **backend:** FastAPI app container, runs uvicorn - **nginx:** Nginx container that serves the static frontend (HTML/JS) and reverse proxies API requests to the backend

The Nginx config routes: - /api/* backend:8000 (FastAPI) - /* static files (the frontend)

This is a multi-stage build the frontend is built in a Node stage, the static files are copied into the Nginx image. The FastAPI container and Nginx container communicate on an internal Docker network. Externally, only port 80 is exposed.

Benefits: - Nginx handles SSL termination, compression, rate limiting - Backend only accessible internally not directly exposed - Multi-stage build keeps the final image small"

Q16. What is Nginx? Why use it in front of FastAPI?

Your Answer: "Nginx is a high-performance web server and reverse proxy. I use it as a reverse proxy in front of FastAPI for several reasons: 1. **Static file serving:** Nginx is much more efficient at serving static HTML/CSS/JS than Python 2. **Load balancing:** If I run multiple FastAPI instances, Nginx distributes requests 3. **SSL termination:** HTTPS is handled at Nginx level FastAPI sees plain HTTP 4. **Rate limiting:** Nginx can rate-limit requests before they hit Python code 5. **Buffering:** Nginx buffers slow client connections so FastAPI can finish and move on Without Nginx, FastAPI (uvicorn) would need to handle all of this itself."

SECTION 7 OCR, EMBEDDINGS, LLM

Q17. What is Tesseract OCR? Why did you integrate it?

Your Answer: "Tesseract is an open-source OCR (Optical Character Recognition) engine maintained by Google. It converts images of text into machine-readable text. I integrated it as a fallback for document

ingestion when a PDF contains scanned pages (images) instead of actual text, standard PDF parsers like PyPDF2 return empty strings. Tesseract processes those pages as images and extracts the text.

The flow:

```
import pytesseract
from PIL import Image
import fitz  # PyMuPDF

def extract_text_with_ocr_fallback(pdf_path):
    doc = fitz.open(pdf_path)
    text = ''
    for page in doc:
        page_text = page.get_text()
        if len(page_text.strip()) < 50: # likely scanned
            pix = page.get_pixmap()
            img = Image.frombytes('RGB', [pix.w, pix.h], pix.samples)
            page_text = pytesseract.image_to_string(img)
        text += page_text
    return text
```
```

---

### Q18. What is the SentenceTransformers model you used? How do embeddings work?

**\*\*Your Answer:\*\***

"SentenceTransformers is a Python library that provides pre-trained models for generating sentence embeddings. The model outputs a fixed-size vector (e.g., 384 dimensions) for any input text.

Internally, the model is a BERT-style transformer fine-tuned on sentence pairs with contrastive learning. It learns to place semantically similar sentences close together in vector space and dissimilar ones far apart.

For reranking, I used `ms-marco-MiniLM-L-6-v2` a Cross-Encoder trained specifically on the MS MARCO passage ranking dataset, which is a large human-labeled relevance dataset from Microsoft. This makes it excellent for re-ranking search results."

---

### Q19. What is Groq? Why use it over OpenAI?

**\*\*Your Answer:\*\***

"Groq is an inference provider that runs LLMs (like Meta's Llama 3.1) on custom-designed LPUs (Language Processing Units) instead of GPUs. The key advantage is speed. Groq achieves 500-800 tokens/second inference, compared to OpenAI's ~50-100 tokens/second. For a streaming chatbot, faster inference means lower latency and better user experience.

I used Llama 3.1 70B via Groq. I also built a provider-agnostic LLM factory:

```
```python
class LLMFactory:
    @staticmethod
    def get_llm(provider: str) -> BaseLLM:
        if provider == 'groq':
            return ChatGroq(model='llama-3.1-70b-versatile')
        elif provider == 'gemini':
            return ChatGoogleGenerativeAI(model='gemini-pro')
```

This means switching providers is one config change the agent code is completely decoupled from the LLM provider."

SECTION 8 AI/ML CONCEPTS

Q20. What are faithfulness and relevance metrics in RAG evaluation?

Your Answer: "These are the two key RAG evaluation metrics:

Faithfulness: Is every claim in the generated answer actually supported by the retrieved documents? An answer is unfaithful if the LLM 'hallucinates' facts not present in the context. Score: what fraction of answer claims can be traced back to a retrieved document.

Relevance: Does the answer actually address the user's question? An answer could be perfectly faithful (every word comes from the documents) but still irrelevant if the retrieved documents were off-topic.

I evaluate these using another LLM call (an 'LLM-as-judge' pattern) or using frameworks like RAGAS. A score of 0.97+ means the system almost always generates answers that are both grounded in retrieved facts AND address the question."

Q21. What is the difference between an LLM and a traditional ML model?

Your Answer: "A traditional ML model (like a decision tree or SVM) is trained on labeled data for a specific task spam detection, house price prediction. It can only do that one task.

An LLM (Large Language Model) is trained on massive amounts of text in a self-supervised way (predicting the next token). This gives it emergent capabilities it can do translation, summarization, code generation, Q&A, reasoning without being explicitly trained for each task. LLMs are general-purpose.

In the context of my project, I use the LLM as a reader given context documents, it synthesizes a natural language answer. The retrieval system handles the knowledge lookup the LLM doesn't need to memorize facts, it just needs to read and synthesize."

Q22. What is chunking? How did you chunk documents?

Your Answer: "Chunking is splitting a long document into smaller segments for storage in the vector database. Why? Because: 1. Embedding models have a token limit (e.g., 512 tokens for many models) 2. Retrieving a full 50-page PDF as context would exceed the LLM's context window 3. Smaller chunks are more precise you retrieve only the relevant paragraph, not the whole chapter

I used recursive character splitting with a chunk size of ~512 tokens and a 50-token overlap. The overlap ensures sentences that fall on chunk boundaries aren't lost.

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=512,
    chunk_overlap=50,
    separators=['\n\n', '\n', '.', '']
)
chunks = splitter.split_text(document_text)
```

Chunk size is a tradeoff: too small loses context; too large retrieves irrelevant text and wastes context window."

SECTION 9 SYSTEM DESIGN

Q23. How would you scale this AI system to handle 1000 concurrent users?

Your Answer: "Several layers: 1. **Horizontal scaling of FastAPI:** Run multiple uvicorn workers behind Nginx load balancer. FastAPI is async so each worker handles many requests concurrently. 2. **LLM rate**

limits: Groq has API rate limits. I'd implement a queue (Redis-based) so requests are processed in order, with exponential backoff on rate limit errors. 3. **Caching:** Cache responses for identical or near-identical queries using semantic similarity if two users ask essentially the same question, serve cached answer. 4. **Vector DB scaling:** ChromaDB is in-memory/local for production, switch to Weaviate, Pinecone, or Qdrant which are distributed and handle millions of vectors. 5. **Async I/O:** All LLM calls and DB calls are already async in FastAPI no blocking threads. 6. **GPU inference:** For self-hosted LLMs instead of Groq API, deploy on GPU cluster with vLLM for high-throughput batching."

Q24. What is the difference between synchronous and asynchronous programming? Why does it matter here?

Your Answer: "Synchronous: Each operation waits for the previous one to complete. If a thread is waiting for an API response (I/O), it's blocked doing nothing.

Asynchronous: While waiting for I/O, the thread can handle other requests. Python's async/await achieves this:

```
# Synchronous - blocks while waiting for Groq
response = groq_client.chat(...) # blocks for 2 seconds

# Asynchronous - yields control while waiting
response = await groq_client.achat(...) # other requests can run during these 2 seconds
```

For an AI assistant, LLM API calls are 1-5 seconds. Synchronous means each request ties up a thread for 5 seconds max ~10 concurrent users per worker. Asynchronous means thousands of requests can be 'in-flight' on the same thread, all waiting for I/O simultaneously. This is crucial for streaming applications."

SECTION 10 BEHAVIORAL & COMPARATIVE

Q25. What was the hardest technical challenge in the Agentic RAG project?

Your Answer: "The hardest part was implementing the Reflexion loop correctly without creating infinite cycles. Early versions had bugs where: 1. The Grader would always score low on a specific query type infinite loop application hangs 2. The retry query wasn't actually different from the original same bad results every time

I fixed this by: 1. Adding a hard retry limit (3 retries) 2. Making the Grader pass a specific critique string to state 'answer lacks specific date information' and the Planner uses this critique to generate a genuinely different query 3. Adding exponential backoff between retries to avoid hammering the LLM API

This debugging process taught me that agentic systems fail in non-obvious ways you need explicit guard rails."

Q26. Compare your two projects. Which is more complex and why?

Your Answer: "The Agentic RAG project is technically more complex it involves multiple AI components (LLMs, embeddings, vector DBs, cross-encoders), async programming, and a non-linear graph-based execution flow. The engineering challenge is much higher.

The Course Registration Portal is more complex in terms of business logic 21 database tables, 3 user roles, 28 pages, real-world workflows like admin approval and attendance tracking. The complexity is in the domain modeling and ensuring data integrity.

Both are production-grade systems I deployed both with Docker Compose on real hosting platforms. The key skill that ties them together is systems thinking understanding how all components interact, where failures can occur, and how to design for reliability."

Q27. What would you improve in the Course Registration Portal if you had more time?

Your Answer: "Several things: 1. **API-first architecture:** Currently Flask renders HTML server-side (SSR). I'd refactor to a proper REST API backend and React/Vue frontend for better separation of concerns. 2. **Async tasks:** Email sending and PDF generation block the HTTP request thread. I'd move them to Celery workers. 3. **More sophisticated RBAC:** Currently role is a simple string. I'd implement proper permissions per resource 'can_mark_attendance', 'can_approve_students'. 4. **Audit logging:** Log all admin actions to an AUDIT_LOG table with timestamps and before/after values. 5. **Rate limiting:** Prevent brute-force login attempts using Flask-Limiter. 6. **Comprehensive test suite:** Add pytest test coverage for all routes and database operations."

SECTION 11 GATE & ACADEMICS

Q28. You scored AIR 1515 in GATE DA. Tell me about it.

Your Answer: "GATE Data Science and AI is a highly competitive exam with 40,000+ candidates. AIR 1515 means I was in the top 4% which is how I got admission into M.Tech at VNIT Nagpur, a National Institute of Technology. The exam covers probability, statistics, linear algebra, machine learning, AI, and programming. Scoring well required deep understanding of the mathematical foundations of ML not just using libraries, but understanding why algorithms work."

SECTION 12 QUICK FIRE CONCEPTUAL

Q29. What is the difference between SQL and NoSQL?

Your Answer: "SQL (relational): Structured tables, fixed schema, ACID transactions, JOINS. Best for structured data with complex relationships. My Oracle database is SQL."

NoSQL: Various models (document, key-value, graph, column-family). Flexible schema, horizontal scaling, eventual consistency typically. ChromaDB is a specialized vector database (NoSQL). I'd use NoSQL for high-write, flexible-schema scenarios like user activity logs."

Q30. What is REST API? What are HTTP status codes?

Your Answer: "REST (Representational State Transfer) is an architectural style for APIs using HTTP methods: - GET: Read resource - POST: Create resource - PUT/PATCH: Update resource - DELETE: Remove resource"

Status codes: - 200 OK: Success - 201 Created: Resource created - 400 Bad Request: Client sent invalid data - 401 Unauthorized: Not authenticated - 403 Forbidden: Authenticated but not authorized (RBAC) - 404 Not Found: Resource doesn't exist - 500 Internal Server Error: Server-side bug

In my FastAPI project, I raise HTTPException with appropriate codes:

```
raise HTTPException(status_code=403, detail='Insufficient permissions')
```
```

---

### Q31. Explain Git workflow. How do you manage your projects?

**\*\*Your Answer:\*\***

"I use a feature branch workflow:

- `main` branch is always deployable
- For each feature, I create a branch: `git checkout -b feature/attendance-calendar`
- Commit small, atomic changes with descriptive messages
- Push to GitHub: `git push origin feature/attendance-calendar`
- Create a pull request, review, merge to main

For this project solo, I used: `git add -p` (interactive staging) to commit only related changes, meaningful commit messages like 'Add MERGE-based attendance upsert for concurrent faculty submissions', and git tags for deployment versions."

---

### Q32. What is Docker volume? Why is it important for databases?

**\*\*Your Answer:\*\***

"A Docker volume is a mechanism to persist data outside the container's filesystem. Containers are ephemeral when you stop and remove a container, all data inside is lost. For a database like Oracle XE, this would mean losing all data every time you restart.

A volume mounts a directory from the host machine (or a managed Docker volume) into the container:

```yaml

volumes:

- oracle_data:/opt/oracle/oradata

Now even if the Oracle container is destroyed and recreated, the database files are on the volume and survive. This is critical for any stateful service in Docker."

End of Part 2 Distributed Multi-Agent AI Knowledge Assistant You now have complete coverage of both projects from basic to advanced.